# What this handbook covers

This is a tuning guide for an editor you already use all day, not a tour of features. It targets the things that make a workspace feel slow: startup time, extension overhead, and search that grinds on a large repo.

Each section names the setting or command directly, so you can apply it without hunting through menus.

# Startup performance

Extension Bisect finds the exact extension slowing startup by disabling half of them at a time until the slowdown disappears - faster than disabling one at a time and reloading.

Workspace-specific extension recommendations, combined with disabling unused global extensions, keep a fresh window from loading dozens of activation events it will never use.

```
Command Palette -> Help: Start Extension Bisect
code --status   # prints extension activation timings
```

# Workspace trust and settings sync

Workspace Trust stops untrusted repositories from running tasks, debug configurations, or extension code the moment you open them - the correct default for cloning unfamiliar code.

Settings Sync carries keybindings, snippets, and extensions across machines, but treat it as a starting point: prune per-machine settings (font size, GPU acceleration flags) rather than syncing them blindly.

# Editor performance for large files

Files above a few MB fall back from the full language grammar to a lightweight tokenizer automatically; you can lower or raise that threshold with editor.largeFileOptimizations for logs and generated files.

Disable minimap and bracket-pair colorization per language for generated code - both cost real render time on files with thousands of lines and no reader benefit.

# Search across multi-root workspaces

search.exclude and files.watcherExclude are different settings: the first hides results, the second stops the file watcher from tracking a folder at all. Set both for node_modules-sized directories.

In a multi-root workspace, scope a search to one folder from the search view's folder filter instead of excluding the others globally - keeps the exclude list reusable per project.

# Debugging with launch.json and compound configs

A compound configuration launches a frontend and backend debug session together with one F5, and stopAll ensures stopping one stops both - the correct shape for any client/server project.

preLaunchTask ties a build or watch task to the debug session so you never attach to stale compiled output.

"compounds": [{ "name": "Full Stack", "configurations": ["Server", "Client"], "stopAll": true }]

# Remote development: SSH, Containers, WSL

Remote-SSH and Dev Containers run the extension host next to the code, not on your laptop - this is the fix for indexing that never finishes on a large repo over a slow filesystem.

A devcontainer.json pinned to an exact base image and extension list makes 'works on my machine' reproducible for the next engineer who opens the repo.

# Git integration and merge conflict tools

The built-in 3-way merge editor shows base, incoming, and current side by side and lets you accept per-hunk instead of per-file - use it before reaching for a separate merge tool.

git.autofetch keeps remote refs current so the Source Control view's incoming/outgoing counts are trustworthy, at the cost of a background fetch on an interval.

# Tasks and problem matchers

A problem matcher turns a build tool's raw stderr into entries in the Problems panel with clickable file:line locations - most popular toolchains ship one already ($tsc, $eslint-stylish, $go).

Background tasks (isBackground: true) with a matching begin/end pattern let a watch task run continuously while VS Code still knows when a build cycle finished.

# Snippets and automation

User and workspace snippets take tabstops and placeholders, not just static text - $1, $2, and ${1:default} let a snippet insert a template you then fill in without leaving the keyboard.

A workspace-scoped .code-snippets file travels with the repo, so a shared snippet is available to every contributor without a settings-sync step.

# Profiling extensions with the CPU profiler

The Command Palette's 'Developer: Startup Performance' and the built-in CPU profiler both point at real time spent, not guesses - use them before disabling extensions speculatively.

A profile taken during a slow operation (not at idle) is the only one worth reading; idle profiles mostly show timers and are noise.

# Keeping the extension list lean

Every installed extension is a candidate for background CPU and memory even when its command is never used - audit with the Extensions view's 'Show Recommended' filtered against what you actually invoke.

Prefer workspace recommendations (.vscode/extensions.json) over installing everything globally; a new clone should suggest exactly what the project needs, nothing more.